CENTINEL

Antes de que suene el teléfono

Guía técnica, operación y defensa ante jueces

Sistema de alerta temprana para anticipar incidentes de TI en pequeñas y medianas empresas, reducir falsas alarmas y explicar por qué una señal merece atención.

Documento basado en la implementación del repositorio local. Distingue explícitamente el pipeline del dataset de la integración LoRa experimental para evitar presentar como terminadas capacidades que aún no están conectadas.

Idea del reto

Una PyME suele enterarse de una falla cuando el cliente llama molesto. Centinel observa señales operativas minuto a minuto y busca avisar antes del impacto, sin convertir cada pico normal en una alarma.

1. Resumen ejecutivo

Centinel es un prototipo EWS (Early Warning System) escrito en Python. Su pipeline principal lee métricas históricas del kit Zikit en orden cronológico, detecta tendencias matemáticas que pueden preceder una falla, contrasta cada anomalía con reglas de negocio o con un proveedor LLM opcional, y genera una alerta con tenant, timestamp, tipo, severidad y confianza.

- El detector actual es basado en reglas y regresión lineal sobre ventanas; no es un modelo de aprendizaje supervisado ni afirma predecir cualquier tipo de caída.
- El modo offline permite una demostración reproducible sin credenciales ni llamadas a un LLM.
- El dashboard muestra datos simulados del dataset o avisos LoRa. No debe interpretarse como telemetría física de laptops mientras no se conecte esa fuente.
- La integración serial actual consume líneas de alerta LoRa y las reenvía; todavía no transforma una serie continua de CPU/RAM/disco/latencia en MetricPoint ni reutiliza el filtro contextual del pipeline histórico.

2. Problema y valor para una PyME

El valor no consiste en decir que un servidor está ocupado, sino en detectar un patrón persistente con tiempo para actuar: ampliar capacidad, corregir un pool de conexiones, investigar una fuga de memoria o verificar almacenamiento antes de que falle una operación visible al cliente.

Necesidad operativa	Respuesta de Centinel	Beneficio esperado
Anticipación	Busca pendiente y consistencia antes del colapso.	Más tiempo para investigar y mitigar.
Menos ruido	Combina ventanas, score, cooldown y contexto del negocio.	Las alertas ganan credibilidad.
Explicabilidad	Adjunta métricas, pendiente, R^2 y razonamiento.	El operador puede validar la señal.
Acción	Entrega una alerta JSON a n8n; n8n puede notificar por Telegram.	Acorta la ruta entre detección y responsable.

La reducción real del tiempo de respuesta debe medirse en una evaluación temporal y, posteriormente, en operación. El repositorio todavía no demuestra con un benchmark auditado un número de minutos ahorrados ni una tasa de falsos positivos; este documento no inventa esas métricas.

3. Datos del reto y empresas

El dataset local está en zikit-dataset/zikit-dataset-main/data. Se verificaron 54 carpetas corrida_01 a corrida_54. Cada corrida contiene datos por tenant y directorios _solucion para evaluación del organizador.

Tenant	Actividad de negocio	Implicación de contexto
gracko	Despacho contable	Nómina, cierres y cargas de declaraciones pueden elevar carga.
gramo	Logística de alimentos	Los ciclos de operación y ventanas de rutas cambian la línea base.
kaiku	Tienda en línea de café	Promociones y picos de pedidos pueden parecer degradación.
ttr	Transporte/GPS	Los ciclos de ingestión y procesamiento de ubicaciones importan.

Archivo	Contenido esperado	Uso observado en el código
contexto.md	Empresa, horario, arquitectura y eventos conocidos.	El filtro contextual lee este contexto para tenant/corrida.
metricas.csv	Una fila por minuto con timestamp y cinco métricas.	Fuente de stream_metrics y entrada del detector.

logs.md	Eventos de servidor, firewall y aplicación.	El filtro LLM puede extraer líneas anteriores a la anomalía; offline no usa logs.
todos_metricas.csv	Métricas unificadas y columna tenant.	Existe en el kit; el pipeline implementado lee archivos por tenant, no este CSV unificado.
_solucion/**	Etiquetas de referencia del organizador.	No se usa por el detector; solo la herramienta de evaluación carga una ruta de etiquetas.

Regla de integridad: ninguna etiqueta de _solucion se usa para entrenar, ajustar una predicción en ejecución o decidir una alerta. Para estimar rendimiento se propone reservar corridas completas como holdout y consultar las etiquetas únicamente al final de la evaluación autorizada.

4. Arquitectura y recorrido de una alerta

Etapa	Módulo	Qué hace
1. Ingesta	streaming_simulator.py	Lee CSV fila a fila, parsea fecha/números y aplica forward-fill causal limitado.
2. Detección	trend_analyzer.py	Actualiza un buffer por tenant y ejecuta cuatro detectores de tendencias.
3. Contexto	llm_context_filter.py	Aplica heurísticas offline o consulta Gemini/OpenAI opcional.
4. Decisión	main.py	Exige incidente real y confianza >= umbral antes de despachar.
5. Entrega	n8n_webhook_trigger.py	POST JSON al webhook, con timeout, reintentos y dry-run.
6. Operación	dashboard_server.py + dashboard.html	Expone estado local, controles y feed de alertas.

1. Fila t de metricas.csv → MetricPoint(timestamp, tenant_id, cpu, ram, disco, latencia_ms, errores).
2. MetricPoint → buffer del tenant → detección de patrones solo con las observaciones recibidas hasta t.
3. DetectedAnomaly → veredicto contextual (is_real_incident, confidence, reasoning).
4. Si el veredicto es positivo y supera el umbral → VerifiedAlert → HTTP POST a n8n.
5. n8n puede entregar una notificación de Telegram. El repositorio no llama a Telegram directamente.

5. Ingesta, calidad de datos y causalidad

El lector usa csv.DictReader y procesa cada fila en secuencia. Los campos numéricos vacíos o inválidos se convierten en None. CausalImputer conserva el último valor conocido y lo rellena hasta tres filas/minutos consecutivos; después deja el campo ausente y reporta blackout.

- Una alerta en t no busca filas futuras: el detector solo recibe el MetricPoint actual y un buffer de puntos anteriores.
- La extracción de logs online limita el intervalo a [t-60 minutos, t] y no agrega líneas posteriores a la anomalía.
- Un error aislado en logs no es por sí solo una alerta: los logs son contexto opcional del filtro LLM, no el detector principal.
- Limitación técnica importante: las regresiones eliminan valores None y usan el orden de los valores restantes como índice x. Los huecos cuentan para cobertura, pero no conservan exactamente su separación temporal en la pendiente. Blackouts largos pueden sesgar la tasa estimada; una mejora es ajustar contra timestamps/minutos reales.

6. Detección matemática actual

El TrendAnalyzer mantiene buffers de hasta 60 puntos por tenant. No evalúa hasta tener al menos 15 puntos. Para cada ventana calcula una regresión lineal y su pendiente (cambio por minuto) y R^2 (qué tan bien los datos siguen una recta). Un score compuesto se compara con ANOMALY_SCORE_TRIGGER = 0.65. No hay un clasificador entrenado con las etiquetas.

Tipo	Ventanas/condiciones configuradas	Lectura operativa
Fuga de memoria	RAM; ventanas 30/60 min, pendiente >= 0.08 puntos porcentuales/min, R^2 >= 0.70; requiere 80% de cobertura de 30 min.	Crecimiento sostenido puede agotar memoria.
Fuga de disco	Mismas condiciones que RAM, aplicada a disco.	Crecimiento persistente reduce espacio disponible.
Agotamiento de recursos	Latencia; ventanas >=15 min, pendiente >=2 ms/min, R^2 >=0.70; CPU estable (pendiente <=0.15).	Cola, pool agotado, deadlock o cuello de botella de I/O posible.
Ráfaga de errores	Ventana 15 min, pendiente >=0.15 errores/min, R^2 >=0.40 y cobertura >=70%.	Aumento sostenido de errores de servicio/red.

La puntuación de anomalía es distinta de la confianza contextual. La primera mide fuerza del patrón matemático; la segunda es la opinión estructurada del filtro offline o LLM. No debe describirse la confianza actual como probabilidad estadística calibrada.

El cooldown actual es de 120 minutos por combinación tenant + tipo de anomalía. Ayuda a no repetir avisos durante el mismo episodio, pero puede suprimir una reincidencia real dentro de ese intervalo; requiere validación con las corridas.

7. Contexto de negocio y filtro anti-señuelos

OfflineContextFilter aplica reglas por hora/día y palabras presentes en contexto.md: respaldos nocturnos, snapshots, nómina y cargas de declaraciones. Si ninguna regla descarta el patrón, algunas categorías se consideran reales con confianza fija; otros casos reciben 0.60. El main pipeline despacha solo si el veredicto es real y alcanza el umbral predeterminado 0.75.

Gemini u OpenAI son caminos opcionales. El prompt incluye el contexto, la anomalía y hasta 30 líneas recientes de logs dentro de una ventana de 60 minutos. El modelo devuelve JSON estricto. En la configuración del reto el proveedor por defecto es Google; si falta la API key, la factory cambia a reglas offline.

- El modo offline es reproducible, barato y útil para demo; sus heurísticas son simplificaciones, no un parser completo de calendarios por empresa.
- El LLM aporta interpretación semántica contextual, no prueba causalidad y puede equivocarse. El umbral y las reglas deben evaluarse con corridas no vistas.
- El fallback ante fallo LLM devuelve incidente=true con confianza 0.5; con umbral 0.75 no se despacha desde el pipeline principal. El comportamiento debe explicarse como fail-closed para la notificación, aunque conserva el veredicto para auditoría.
- Enviar logs o contexto a un proveedor externo tiene implicaciones de privacidad. El demo usa datos sintéticos; para producción se necesita autorización y minimización de datos.

8. Salida, n8n y dashboard

El payload de alertas generado por el pipeline principal tiene esta forma:

```
{ "tenant_id": "gracko", "timestamp": "2026-10-08T08:30:00", "anomaly_type": "resource_exhaustion",
"severity": "high", "anomaly_score": 0.82, "llm_confidence": 0.85, "llm_reasoning": "...",
"description": "..." }
```

El payload cubre el mínimo del reto: tenant, timestamp y confianza; además incluye tipo, severidad, score y explicación. Una ejecución CLI guarda alerts.json y run_stats.json debajo de output/. Dry-run evita enviar al webhook real.

Ruta	Método	Función
/api/state	GET	Estado del tenant, métricas/historial, estadísticas y últimas anomalías.
/api/control	POST	Cambiar tenant, velocidad, pausar/reanudar o reiniciar.
/api/lora-alert	POST	Recibir una alerta serial adaptada; no carga una serie de MetricPoint.

El dashboard simula la reproducción de una fila por punto con velocidad ajustable; 50 puntos/segundo significa reproducción acelerada del histórico, no captura real de 50 muestras por segundo. El modo LoRa en vivo espera avisos y deja las métricas del dataset vacías.

9. Qué está integrado de LoRa y qué falta

lora_serial_bridge.py escucha un puerto serial (por defecto COM8) a 115200 baudios y espera una línea ASCII de cuatro campos separada por |:

```
ALERTA_SERVER|SRV_LAPTOP_1|CPU alta|85.0
ALERTA_SERVER|SRV_LAPTOP_2|Latencia alta|120.0
```

El receptor Heltec compartido por el equipo decodifica JSON LoRa con las claves id, msg y temp; si el id empieza con SRV_, imprime esa línea serial. Ese formato coincide con el parser Python. La comparación empieza por SRV_; por tanto, laptop_1 sin prefijo no cumple ese filtro del firmware.

Funciona en el código actual	No debe afirmarse todavía
Serial → parseo de alertas → dashboard/n8n; conserva source_device y valor.	No existe captura de CPU/RAM de Windows integrada en las laptops emisoras.
El dashboard acepta mensajes de prueba para laptop_1/laptop_2.	El detector de tendencias histórico no procesa las alertas seriales.
Un mensaje HELTEC ALERTA_SERVER es compatible con el parser.	No está demostrada recepción estable de laptop_1 y laptop_2 en operación.
El dashboard ofrece modo de espera LoRa.	No hay métricas continuas reales ni adaptación LoRa a MetricPoint/OfflineContextFilter.

Riesgo en el firmware receptor proporcionado: loopLoRa devuelve temprano si radio.readData falla, el mensaje está vacío, el JSON no parsea o el id no empieza por SRV_. En esos caminos no ejecuta radio.startReceive() al final. Si RadioLib requiere rearmar recepción tras cada lectura, puede quedarse sin recibir. La implementación robusta debe rearmar recepción en todas las salidas y comprobar el código de retorno.

El bridge actual marca cada aviso recibido como incidente confirmado y lo envía directamente a dashboard/n8n; no aplica el cooldown de 120 min del TrendAnalyzer, sino un cooldown serial separado (300 s por emisor y descripción). El valor llamado temperature no siempre es temperatura: depende de lo que el firmware coloque en temp.

Comandos de ejecución en el PC receptor (PowerShell):

```
Set-Location 'C:\Agente SRE'
python .\dashboard_server.py 8080
python .\lora_serial_bridge.py --port COM8 --dry-run
```

Usar --dry-run mientras se valida recepción evita notificaciones reales a n8n. Confirmar el COM asignado en el Administrador de dispositivos; si el receptor está en otro PC, 127.0.0.1 no apunta al dashboard central y hay que parametrizar la URL de destino y la conectividad de red.

10. Cómo ejecutar y demostrar

Demo del análisis histórico, sin emitir alertas externas

```
Set-Location 'C:\Agente SRE'
python .\main.py --tenant gracko --offline --dry-run
python .\main.py --corrida 48 --tenant gracko --offline --dry-run
```

Repetir sobre un tenant/corrida por vez y presentar el log de la alerta: tipo, timestamp, score, pendiente, R^2, veredicto, confianza y motivo. Para una demostración de validación, fijar de antemano corridas de desarrollo y holdout; no inspeccionar _solucion hasta haber congelado el detector.

Demo del dashboard simulado

```
Set-Location 'C:\Agente SRE'
python .\dashboard_server.py 8080
Abrir http://localhost:8080
```

La opción normal del selector reproduce el dataset. La opción “LoRa en vivo” es solo el feed de alertas recibidas, no un gráfico de métricas de laptop.

Chequeo serial

```
Get-CimInstance Win32_SerialPort | Select-Object DeviceID, Name
python .\lora_serial_bridge.py --port COMx --dry-run
```

El resultado esperado es que la consola registre el emisor exacto y que el feed lo muestre. Una línea que no cumpla ALERTA_SERVER|source|message|valor se ignora.

11. Evaluación: qué medir y qué no afirmar

La métrica objetivo del reto requiere matching temporal de alertas contra incidentes: recall de incidentes reales detectados, precisión/false alarms, y tiempo de anticipación respecto al impacto. Separar corridas completas en desarrollo y prueba evita elegir umbrales mirando el mismo caso que se reporta.

Métrica	Definición para la presentación
Recall	Incidentes reales con al menos una alerta elegible / total de incidentes reales.
Precision	Alertas válidas que corresponden a incidente / total de alertas emitidas.
Falsas alarmas	Avisos sin incidente real correspondiente; reportar conteo y tasa por unidad temporal.
Lead time	Minutos entre alerta y timestamp de impacto; positiva si anticipa.
Cobertura de evaluación	Tenants/corridas usados, exclusiones, umbrales fijados antes del holdout.

La herramienta evaluate_competition.py es preliminar y no se debe usar para anunciar resultados oficiales sin corregirla: su scorer no comprueba que alerta e incidente pertenezcan al mismo tenant ni al mismo tipo, puede contar varias alertas como verdaderos positivos para un mismo incidente, y carga una ruta general _solucion que no distingue la corrida. Aunque separa etiquetas reales y señuelos, el matching no utiliza esa lista de señuelos explícitamente. Por ello este documento no presenta scores calculados por ese script.

Protocolo recomendado: agrupar corridas completas para ajuste y holdout; hacer matching uno-a-uno por tenant/tipo con una ventana temporal del reglamento; contar duplicados y señuelos como falsos positivos según regla oficial; informar mediana/promedio de anticipación y dispersión, además de precisión/recall por tenant.

12. Preguntas probables de jueces (y respuestas honestas)

¿Qué problema concreto resuelve y para quién?

Reduce la brecha entre una señal técnica y la primera queja del cliente de una PyME. Detecta precursores en RAM, disco, latencia y errores y entrega un aviso accionable al operador antes del posible impacto.

¿Cómo demuestra que alerta antes y no después?

Cada punto se procesa en orden y las ventanas son right-aligned al timestamp actual. Se informa el timestamp de la alerta y se compara contra el impacto solo en un holdout de corridas; aún no presentamos una cifra de anticipación validada por el scorer actual.

¿Dónde está el machine learning?

El núcleo del detector no es ML entrenado: son detectores explicables basados en OLS, umbrales de pendiente/R^2 y reglas contextuales. El LLM es opcional y clasifica contexto, no se entrena con etiquetas del evaluador.

¿Por qué no alertar cuando CPU supera un umbral?

Una foto puntual suele confundir carga normal con degradación. Se busca dirección, persistencia y forma de tendencia; por ejemplo, latencia que sube con CPU estable puede sugerir cola o pool agotado.

¿Cómo evitan backups o nómina como falsas alarmas?

El filtro lee contexto.md y aplica reglas por ventana horaria y tipo de patrón. Es un filtro inicial y debe medirse por corrida; no garantizamos que cubra cada calendario posible.

¿La confianza 0.85 es 85% de probabilidad real?

No. Hoy es el confidence devuelto por reglas o LLM; no está calibrado estadísticamente. Debe calibrarse con validación separada antes de interpretarlo como probabilidad.

¿Cómo manejan celdas vacías y apagones?

Forward-fill hasta 3 minutos y después None. El detector exige cobertura mínima. Reconocemos que la regresión omite None sin preservar la distancia temporal exacta; timestamps reales en x es una mejora prioritaria.

¿Usaron las soluciones del evaluador?

El predictor no accede a _solucion. Para un benchmark, las etiquetas deben abrirse solo después de congelar la versión evaluada. El script local de scoring es preliminar y no se presenta como resultado oficial.

¿Qué hacen los 54 runs?

Permiten probar variabilidad por corrida. La práctica correcta es separar corridas completas en ajuste y holdout, y reportar resultados por tenant y globales.

¿Los LoRa ya transmiten telemetría de las laptops al detector?

No completamente. Hay un receptor serial de alertas de texto y una ruta a dashboard/n8n. Todavía falta agente de captura por laptop, stream de métricas, rearmado fiable del receptor y adaptar esas métricas al mismo MetricPoint/TrendAnalyzer.

¿Qué pasa si se cae internet, el LLM o n8n?

El pipeline offline evita depender del LLM; el envío al webhook tiene timeout y reintentos. La alerta no puede llegar por Telegram si n8n/red fallan. Se necesita cola durable y replay para operación productiva.

¿Cuánto tiempo de anticipación ya garantizan?

No prometemos un lead time universal. Depende del incidente, de cuándo la señal cruza los umbrales y de la ventana; se debe medir por incidente en corridas no usadas para ajustar.

13. Riesgos y hoja de ruta

Prioridad	Trabajo	Criterio de salida
P0	Corregir evaluación temporal uno-a-uno por tenant y corrida.	Métricas reproducibles, falsos positivos y lead time con holdout.
P0	Rearmar LoRa RX tras toda rama de lectura y probar pérdida/JSON inválido.	Receiver reanuda RX siempre; pruebas de hardware documentadas.
P0	Implementar agente de métricas en laptop_1/2 y protocolo versionado.	CPU/RAM/disco/latencia/errores con timestamp y tenant identificables.
P0	Adaptar LoRa stream al MetricPoint, detector y contexto común.	Alerta LoRa entra al mismo pipeline, no se autoclasifica sin filtro.
P1	Ajustar regresión usando timestamps y calcular cobertura de huecos.	Pendientes correctas frente a blackouts y muestreo irregular.
P1	Calibrar confianza y tuning usando solo partición de desarrollo.	Umbral fijado antes de holdout y curva precisión-recall.
P1	Agregar persistencia/cola idempotente y estado de salud de fuentes.	Reinicio sin pérdida y alertas de sensor desconectado.
P2	Agregar autenticación, TLS y controles de acceso al dashboard.	Endpoint no expuesto sin protección en una red productiva.

14. Seguridad y operación

- Las API keys deben estar en variables de entorno, no en el código o el documento.
- El servidor dashboard escucha en 0.0.0.0 y la API no implementa autenticación: mantenerlo en red local controlada; no publicarlo directamente a Internet.
- Validar límites y formato del JSON/serial, restringir tamaño de mensajes y definir autorización si el endpoint LoRa se expone fuera del host.
- Mantener dry-run para demostración; separar credenciales, webhook de prueba y destino de producción.

15. Mapa rápido del repositorio

Archivo	Responsabilidad
config.py	Rutas, thresholds, tenants, LLM y webhook.
models.py	MetricPoint, TrendResult, DetectedAnomaly, LLMVerdict y VerifiedAlert.
streaming_simulator.py	Parser CSV, stream causal e imputación.
trend_analyzer.py	Buffer, regresión, detectores y cooldown.
llm_context_filter.py	Contexto, extracción de logs, provider LLM y heurísticas offline.
n8n_webhook_trigger.py	Payload webhook, dry-run, timeout y reintentos.
main.py	CLI y orquestación de pipeline por tenant/corrida.
dashboard_server.py	Servidor HTTP/API y simulación del dashboard.
dashboard.html	Interfaz Centinel y modo LoRa en vivo.
lora_serial_bridge.py	Adaptador actual de alertas LoRa seriales.
evaluate_competition.py	Scorer preliminar: no usar sus cifras como resultados oficiales sin corregir.
build_project_document.py	Generador de este documento Word.

Cierre para la presentación

Centinel propone pasar de reaccionar a la queja a investigar una señal antes del impacto. Su valor técnico es combinar tendencias explicables y contexto del negocio; su valor de producto es enviar una alerta comprensible a quien puede actuar. La defensa más sólida ante jueces es demostrar el flujo, reconocer qué está integrado, no filtrar etiquetas al predictor y reportar métricas solo después de una evaluación temporal limpia.